Source: docs\STOREFRONT_LOADING.md | Export: 11 November 2026

Storefront loading and recovery

Implemented locally on 1 November 2026. This work does not publish application code, rules or indexes, and does not change production records.

Presentation

`src/loading/` provides decorative skeletons, one accessible loading announcement per section, a five-second slow-request message, recovery panels, retained-content updating feedback and bounded image placeholders. Reduced-motion placeholders are static. Placeholders contain no invented products, prices, stock or account facts.

Coverage includes homepage banners, collection thumbnails, product grids and campaigns; Shop/search/category results; product gallery/details; Wishlist and Bag; checkout; customer session initialization, profile, addresses and points; My Orders and order details/confirmation; About, Contact and policy content. Contact inputs and usable static navigation remain available while their content section loads.

Homepage banner geometry remains 8:3 on desktop and 4:5 below 768px. Product placeholders use 3:4 artwork. Image loading is separate from data loading, with an intentional unavailable-image fallback. Lazy-image deadlines begin near the viewport, so images below a long list are not failed before their download starts. Existing approved auth portrait fallback remains available independently of appearance-settings loading.

Defects corrected

- Content and campaign refresh failures previously cleared loaded banners/items. Refreshes now retain successful data for the same resource identity.
- Catalog refresh/pagination failures could hide valid results. Failed pagination retains its cursor and loaded products; Retry requests the failed page. Background refresh restores the already loaded extent with fresh cursors.
- Customer request helpers used an external cancellation signal *instead of* a timeout. A supplied navigation signal now retains its deadline. Shared reads additionally settle even when an underlying SDK promise ignores cancellation.
- Category cache notifications could repeatedly reset the server-check deadline and replace loaded categories with loading text. The check is bounded and successful public categories remain visible with a stale indication.
- Address/shopping failures could discard confirmed displayed state. Reads now coalesce; reconnection during an active read schedules a guarded follow-up read instead of losing the event.
- Checkout could stay loading indefinitely after an address/Bag dependency failed. It now exposes recovery and Retry refreshes the failed dependency. A retained review cannot enable continuation until a fresh valid review succeeds.
- Session restoration now has a recovery deadline. A transient session-refresh error retains the known customer identity and mounted customer content; actual sign-out/account changes still clear private state. Server authorization remains authoritative.
- Profile focus/reconnection refresh uses a read-only server document read instead of repeating profile initialization/directory synchronization writes. Existing explicit profile-save recovery remains intact.

State and retry contract

Initial loading, a slow pending request, successful content, background updating, confirmed empty results, offline with content, offline without content and online failures have separate presentation. Errors never become successful empty responses or permanent skeletons. Old-query products are not displayed as results for a new query.

Browser connectivity events are hints. Offline content is labelled potentially stale; an online event initiates read revalidation, and subsequent API failures still show recovery. An online event does not establish fresh prices, stock or order status.

Shared public reads settle within 15 seconds; slow guidance begins after five seconds. Customer API deadlines remain 20/30 seconds, with token acquisition bounded to 15 seconds and shared read wrappers using 15 seconds where appropriate. Initial session restoration uses ten seconds. Images use a 15-second visible download deadline. Timers, listeners, controllers and stale-result guards are cleaned up on navigation/unmount.

Memory is scoped to mounted resources and exact query/customer identities. Existing customer order-history return context and checkout-intent storage contracts are preserved. No new persistent private-data cache, service worker, PWA, offline write queue or automatic mutation replay is added. Existing operation IDs and uncertain-acknowledgement reconciliation remain in place. Administrator checks, server handlers, Firestore rules and business calculations are unchanged.

Local validation

- `npm run lint`: TypeScript passed.

- `npm test`: 160 tests passed, including deadline/cancellation/error-preservation and skeleton accessibility tests.
- `npm run build`: passed; Vite's existing large-chunk advisory remains.
- `npm run test:loading`: dedicated loopback Auth/Firestore suite and controlled Chromium network acceptance. Six emulator groups passed: category rules, shopping, delivery, orders, customer history and discovery (seven test cases).
- Controlled browser checks passed at 320, 390, 768, 1440 and 1920px: independent initial loading; slow data/images; offline before data and after content; reconnection; online 503 responses; finite hanging requests; pagination failure/retry; new-query isolation; private initial states; retained Bag results; and no mutation replay.
- Existing discovery, banner timing, customer history, Shopping Bag/Wishlist and customer order-state browser regressions passed. Checkout design regression passed with additional held-review, online failure, retained typed coupon, disabled continuation and offline/reconnect assertions. Existing receipt/idempotency, account isolation, cancellation eligibility, price acknowledgement and stock-blocking checks remain active.
- Fixtures and image responses are synthetic/local. Expected emulator transport warnings occur during deliberately offline tests. They are not evidence of production connectivity.

The suite supports `PARINAYA_LOADING_BROWSER_ONLY=true`, `PARINAYA_LOADING_REGRESSIONS_ONLY=true` and a comma-separated `PARINAYA_LOADING_FILES` selection to rerun a focused subset without repeating already passed checks. These are test-runner switches, not storefront configuration.

Screenshots

Captured with local fixtures and reduced motion, under `.cache/loading-design/`:

State	Mobile	Desktop
Initial skeleton	`home-skeleton-390.png`	`home-skeleton-1440.png`
Offline without data	`offline-390.png`	`offline-1440.png`
Recovered content	`recovered-390.png`	`recovered-1440.png`

The same states are captured at all five widths. Private Bag/Wishlist/orders/profile-section skeletons are in that directory. Checkout loading captures are in `.cache/checkout-design/loading-{width}.png`; its existing three-stage captures remain available. The recovered homepage photograph/product data are test fixtures, not a production content change.

Limits and hosted acceptance

There is no new first-load offline website support. If HTML/JavaScript is not already available to the browser, React cannot render this recovery UI. A cold offline reload is not guaranteed to work. Native browser/Firebase cache availability is not an offline-read guarantee.

Production hosting, real-device networking and deployed API/index/rules readiness were not tested or changed. After the owner releases the application changes, check the hosted storefront with slow/offline networking and an existing customer session. No rules/index publication or new environment variables are required by these presentation changes.